

SPM: Models of Computation

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Goal: understand limits of parallelism and cost of communication
- Work-Span model
 - Limits of parallelism, concepts of work and critical path
- PRAM model
 - Idealized shared-memory algorithm design
- BSP model
 - Concrete model showing cost of communications and synchronizations

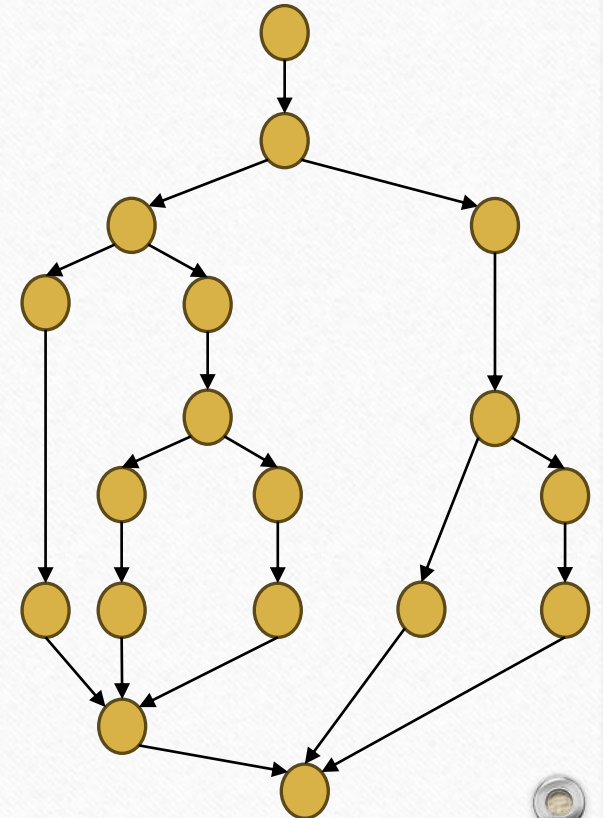
Why high-level abstract models?

- Models help us understand what is possible before worrying about how
- Separate algorithm design from machine details
- Reason about scalability and fundamental limits
- Focus on the algorithm first, then map to real systems
- Provide **lower bounds** on achievable performance
 - A lower bound sets a limit on the best performance any algorithm can achieve for a problem

Work-Span model

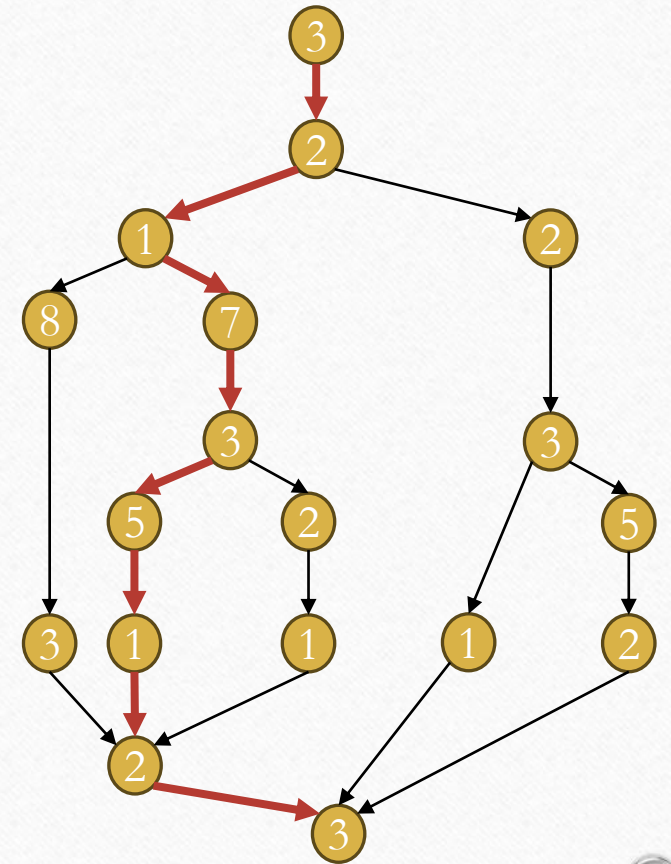
The **Work-Span model** (also called **Work-Depth**) model provides more strict bounds than those offered by the Amdahl's and Gustafson's laws

- A parallel program can be represented as a DAG (Directed Acyclic Graph)
 - Nodes are tasks and edges are data dependencies between tasks
 - A task is a unit of work, i.e., **arbitrary sequential code**
- A task can be executed (ready) iff all its predecessors in the graph are completed
- Model assumptions:
 - p identical processors, each executing one ready task at a time
 - **Greedy scheduling**, i.e., whenever there is a ready task (and an available processor), the task is executed immediately
- **Parallelism is determined by the structure of the DAG**



Work-Span model: terminology

- T_1 is the total **work**
- T_∞ is the **span** (and also called **critical path**)
 - Longest chain of dependencies with unit task
 - The path from the root to on leaf with the highest associated cost
- T_1 measures total computation, T_∞ measures inherent sequential computation
- In the example DAG we have $T_1 = 54$ and $T_\infty = 27$



Work-Span model: lower bounds

T_p is the execution time using p processors under greedy scheduling

$$T_p \geq \frac{T_1}{p}$$

Work bound

Ideal case: perfect load balancing and zero overhead

The best time is $\frac{T_1}{p}$

$$T_p \geq T_\infty$$

Span bound

Execution time cannot be smaller than the critical path

Parallel execution time is limited by total work and by the critical path

Work-Span model: speedup limits

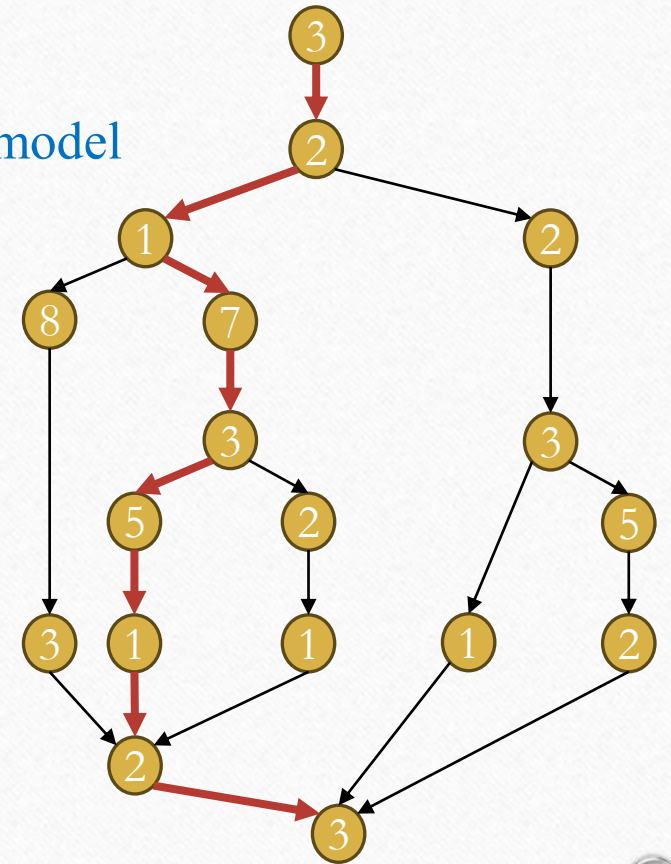
$$S(p) = \frac{T_1}{T_p} \leq \frac{T_1}{\frac{T_1}{p}} = p$$

Superlinear speedup not possible in the model

$$S(p) = \frac{T_1}{T_p} \leq \frac{T_1}{T_\infty}$$

Maximum speedup is bounded by the available parallelism T_1/T_∞

$$T_1 = 54 \text{ and } T_\infty = 27 \rightarrow S(p) \leq 2$$



Brent's theorem (1974)

- Assume a parallel computer whose processors can perform a task in unit time with greedy scheduling of tasks
- Assume that the computer has enough processors to exploit the maximum concurrency in an algorithm containing T_1 tasks such that it completes in T_∞ time steps
 - At each level i of the DAG, there are m_i tasks (i.e., $\sum_{i=1}^n m_i = T_1$)
 - We may use m_i processors to compute all results at level i in $O(1)$
- **Brent's theorem** states that a similar computer with fewer processors p can execute the algorithm with the following upper time limit:

$$T_p \leq \frac{(T_1 - T_\infty)}{p} + T_\infty$$

Parallel time is bounded by the distributed work plus the critical path

- Brent's inequality provides an **upper bound** on T_p

Brent's theorem: proof sketch

$$T_p \leq \frac{(T_1 - T_\infty)}{p} + T_\infty$$

Proof sketch:

- The DAG has n levels, and level i contains m_i unit tasks
- At each level there is at least a unit task to compute, i.e., $n = T_\infty$
- At each level i , the time taken by p processor is:

$$T_p^i = \left\lceil \frac{m_i}{p} \right\rceil \leq \frac{m_i + p - 1}{p}$$

- then summing over all n levels

$$T_p = \sum_{i=1}^n T_p^i \leq \sum_{i=1}^n \left(\frac{m_i + p - 1}{p} \right) = \frac{1}{p} T_1 + \frac{(p-1)}{p} T_\infty = \frac{(T_1 - T_\infty)}{p} + T_\infty$$

Remember that:

$$\left\lceil \frac{x}{y} \right\rceil \leq \frac{x + y - 1}{y}$$

Implication of Brent's theorem

- Considering the speedup, we have that

$$S(p) \leq \min(p, \frac{T_1}{T_\infty})$$

- The speedup is bounded by both the number of processors and the available parallelism
- To obtain good speedup, T_1 must be significantly larger than T_∞ (i.e., $T_\infty \ll T_1$). In this case $(T_1 - T_\infty) \approx T_1$, therefore, from Brent's inequality we have

$$T_p \approx \frac{T_1}{p} + T_\infty \quad \text{if } T_\infty \ll T_1$$

- Algorithm design implications: we should focus on reducing the **span**, because the span is the fundamental asymptotic limit on scalability. We could also increase the work but only if it enables a drastic decrease in the span
- Overall, we have:

$$\frac{T_1}{p} \leq T_p \leq \frac{T_1}{p} + T_\infty$$

Implication of Brent's theorem (cont.)

$$S(p) = \frac{T_1}{T_p} \geq \frac{T_1}{\frac{T_1}{p} + T_\infty} = \frac{p}{1 + p \frac{T_\infty}{T_1}}$$

$$S(p) \approx p \quad \text{if} \quad \frac{T_1}{T_\infty} \gg p$$

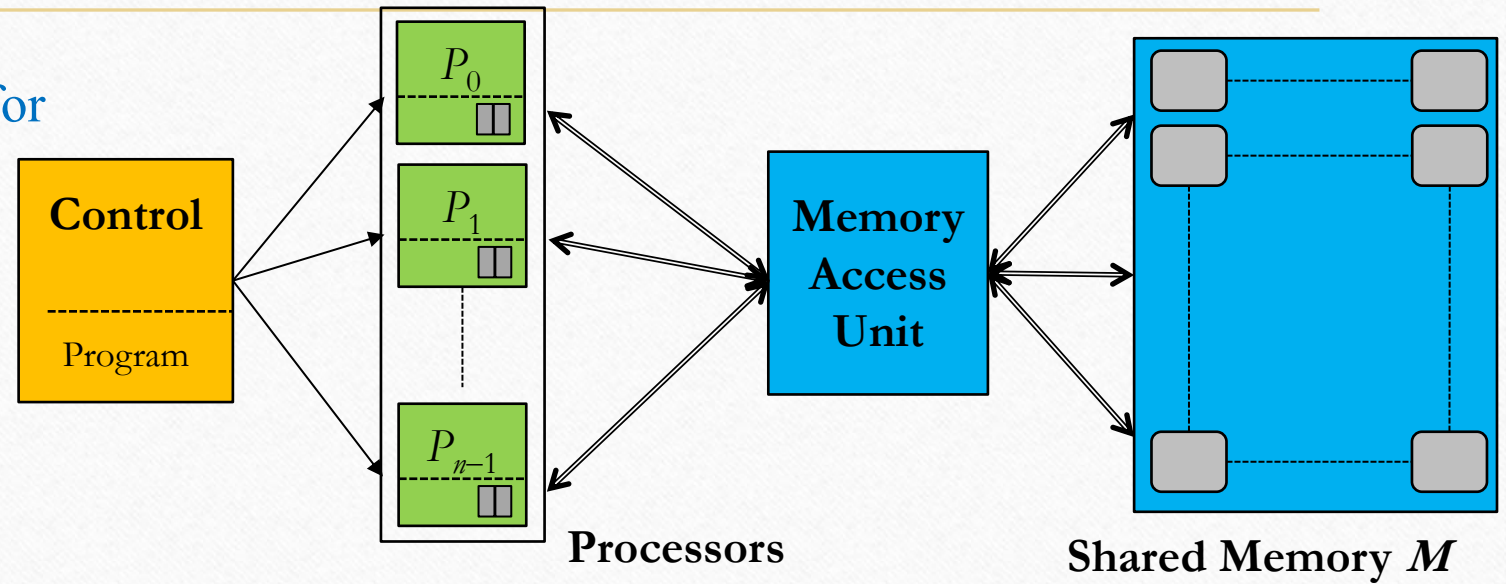
- It says that greedy scheduling achieves (almost) linear speedup if a problem is **overdecomposed** to create much more potential parallelism than the number of processors
- The excess parallelism is called the **parallel slack**, and is defined as $\frac{T_1}{T_\infty}$
- In practice, it has been observed that a parallel slack of at least 8-10 is often sufficient

Limits of the Work-Span model

- These bounds rely on idealized assumptions:
 - **No parallel overhead**
 - the parallel execution performs only T_1 total operations
 - **The memory bandwidth is not a bottleneck**
 - we assume PRAM-like abstract machines where memory access is assumed not to limit performance
 - **Greedy scheduling**
 - Ready tasks are executed immediately, with no delays due to lock or synchronization
- These are scalability bounds under ideal assumptions. They help us reason about limits, but they do not predict the exact runtime of a real implementation

Parallel Random Access Machine - PRAM

- Idealized Shared-Memory model for algorithm design
- Idealized platform
 - No caching, no NUMA
 - No synchronizations overhead



- n processors $P_0, \dots, P_{n-1}$ are connected to a global shared memory M
- Any memory location is uniformly accessible from any processor in constant time ($O(1)$)
- Communication between processors can be implemented by reading and writing to the globally accessible shared memory
- Synchronous execution model (**lockstep execution**)

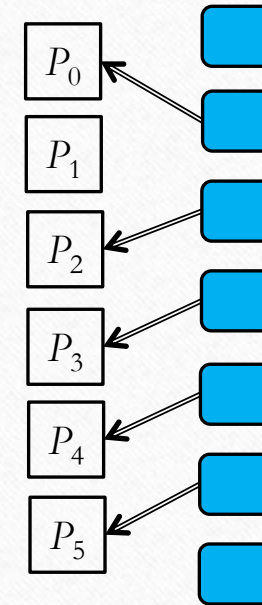
PRAM

- n identical processors $P_i, i = 0, \dots, n-1$ **operate in lockstep**
- In every step, each processor executes an instruction cycle in three phases:
 1. **Read phase:** Each processor can simultaneously read a single data item from a (distinct) shared memory cell and store it in a local register.
 2. **Compute phase:** Each processor can perform a fundamental operation on its local data and store the result in a register.
 3. **Write phase:** Each processor can simultaneously write a data item to a shared memory cell, whereby the exclusive write PRAM variant allows writing only distinct cells while the concurrent write PRAM variant also allows processors to write to the same location (possible race conditions).
- **Uniform complexity analysis:** each step on the PRAM takes $O(1)$ time

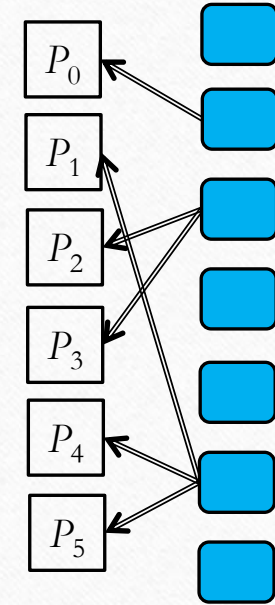
PRAM Variants

- PRAM variants differ in how they handle concurrent memory accesses
- Exclusive Read Exclusive Write (EREW):** No two processors are allowed to read or write to the same shared memory cell during any cycle
- Concurrent Read Exclusive Write (CREW):** Several processors may read data from the same shared memory cell simultaneously. Still, different processors are not allowed to write to the same shared memory cell

EREW



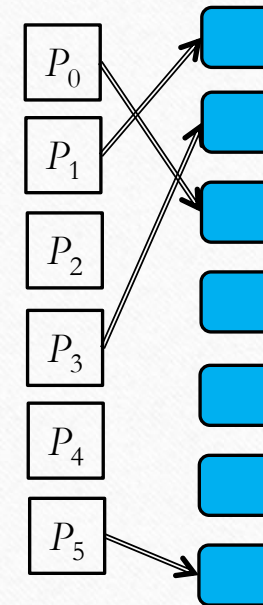
CREW



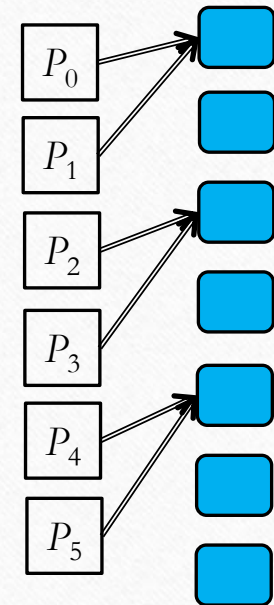
PRAM (more) Variants

- **Concurrent Read Concurrent Write (CRCW):** Both simultaneous reads and writes to the same shared memory cell are allowed. In case of a simultaneous write we further specify which value will actually be stored:
 - **Priority CW:** Processors have been assigned distinct priorities, and the one with the highest priority succeeds in writing
 - **Arbitrary CW:** A randomly chosen processor succeeds in writing its value.
 - **Common CW:** If the values are all equal, then this common value is written, otherwise, the memory location is unchanged.
 - **Combining CW:** All values to be written are combined into a single value by means of an associative binary operation (e.g. sum, product, minimum, logical OR/AND)

CREW



CRCW



Complexity Measures in PRAM

- PRAM ignores real hardware costs to simplify parallel algorithm design
- Complexity analysis for PRAM algorithms involves two measures:
 - **Time complexity ($T(n)$)**: It considers the number of synchronous steps needed to complete the algorithm
 - **Processor complexity ($P(n)$)**: It considers the number of processors used to execute the algorithm
- The cost of the algorithm is

$$C(n) = T(n) \times P(n)$$

- A PRAM algorithm is cost-optimal if its cost matches the best sequential complexity asymptotically

Prefix Computation

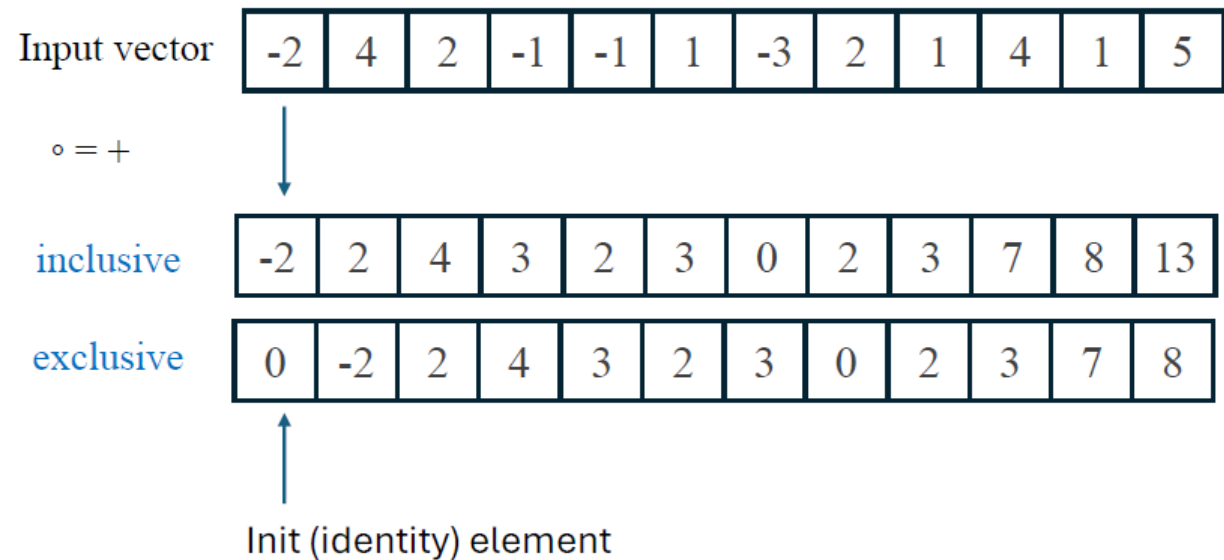
- Binary **associative** operation $\circ$ on the set X ; $\circ : X \times X \rightarrow X$
 - $(x_i \circ x_j) \circ x_k = x_i \circ (x_j \circ x_k)$ for all $x_i, x_j, x_k \in X$.
 - Examples: Addition, Multiplication, Minimum, Maximum, String concatenation, boolean AND/OR
- $X = \{x_0, \dots, x_{n-1}\}$, $x_i \in X$ for all $i = 0, \dots, n-1$. We want to compute
 - $s_0 = x_0$, $s_1 = x_0 \circ x_1$... $s_{n-1} = x_0 \circ x_1 \circ \dots \circ x_{n-1}$.
 - In other words, $s_0 = x_0$ and $s_i = s_{i-1} \circ x_i$ for $i = 1, \dots, n-1$
- Obtaining $S = \{s_0, \dots, s_{n-1}\}$ from $X = \{x_0, \dots, x_{n-1}\}$ is called **prefix computation**

Prefix Computation example

- Example: Let's consider as $\circ$ the min operator
 - Input: {39, 21, 20, 50, 13, 18, 2, 33, 49, 39, 47, 15, 30, 47, 24, 1}
 - Output: {39, 21, 20, 20, 13, 13, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1}
 - Each output element is the minimum of all previous input elements up to that position
- Lower bound: $\Omega(n)$
- The algorithm implementing the prefix computation is often called **scan**

Exclusive and Inclusive Scan

- Two versions of the scan algorithm: **inclusive** and **exclusive**
- The C++ STL library provides `std::inclusive_scan` and `std::exclusive_scan`
- **Prefix computation usually refers to the inclusive version**



Parallel Prefix ($\circ = '+'$) on a PRAM

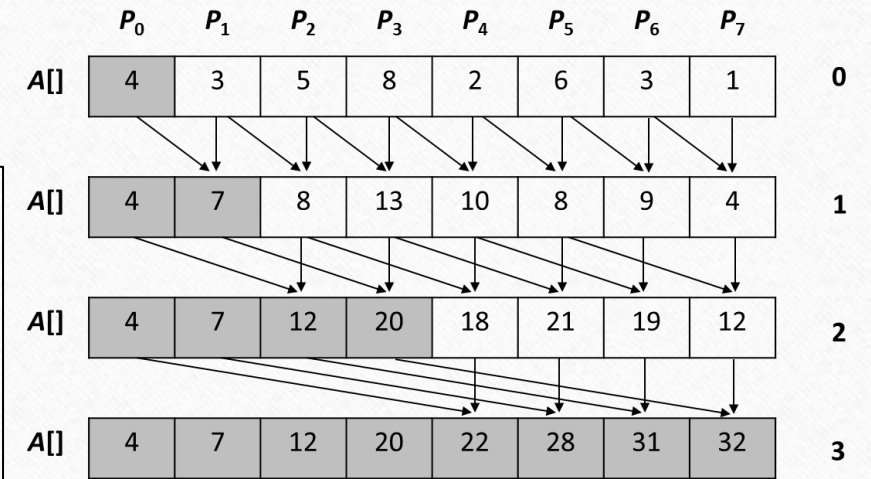
- Goal: design a cost-optimal PRAM algorithm, i.e., $C(n) = O(n)$
- We use the *recursive doubling algorithm* using $p=n$ processors

```

for (j = 0; j < n; j++) do_in_parallel           // each processor j
    reg_j = A[j];                               // copies one value to a local register
for (i = 0; i < ceil(log(n)); i++) do           // sequential outer loop
    for (j = pow(2, i); j < n; j++) do_in_parallel // each processor j
        reg_j += A[j - pow(2, i)];              // performs computation
        A[j] = reg_j;                          // writes result to shared memory
    }

```

- $C(n) = T(n, p) \times p = O(\log n) \times n = O(n \times \log n)$
- This algorithm is fast but **NOT** cost-optimal when using $p=n$ processors.



$O(\log n)$ steps

Each element of the final sequence represents the sum of all preceding elements in the original sequence

Parallel Prefix ($\circ = '+'$) on a PRAM

- To reduce the cost $C(n)$ we can reduce the number of processors used (i.e., $P(n)$)
- Let's use $p = n/\log(n)$ processors and the following algorithm:
 1. Partition the n input values into chunks of size $\log(n)$. Each processor computes local prefix sums of the values in one chunk in parallel (takes time $O(\log(n))$)
 2. Perform the old non-cost-optimal prefix sum algorithm on the $\frac{n}{\log(n)}$ partial results (takes time $O(\log(n/\log(n)))$)
 3. Each processor adds the value computed in step 2 by its left neighbor to all values of its chunk (takes time $O(\log(n))$)
- Same asymptotic time, fewer processors

Parallel Prefix ($\circ = '+'$) on a PRAM

P0	0	1	2	3
P1	4	5	6	7
P2	8	9	10	11
P3	12	13	14	15

P0	0	1	3	6
P1	4	9	15	22
P2	8	17	27	38
P3	12	25	39	54

- $$C(n) = T(n,p) \times p =$$

$$O(\log n) \times O(n/\log(n)) = O(n)$$

Thus, **this algorithm is cost-optimal**

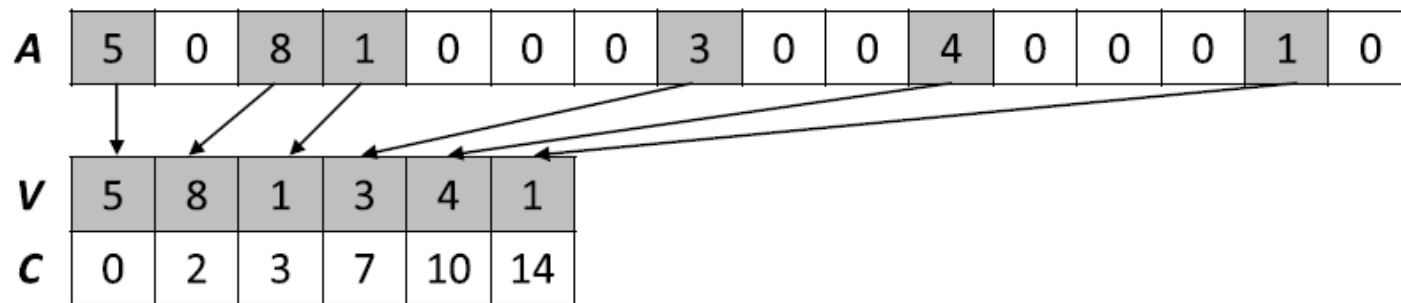
P0	0	1	3	6
P1	4	9	15	28
P2	8	17	27	66
P3	12	25	39	120

P0	0	1	3	6
P1	10	15	21	28
P2	36	45	55	66
P3	78	91	105	120

NOTE: We did not beat the $O(\log n)$ time asymptotically, but we reduced the number of processors enough to bring the total cost back to $O(n)$

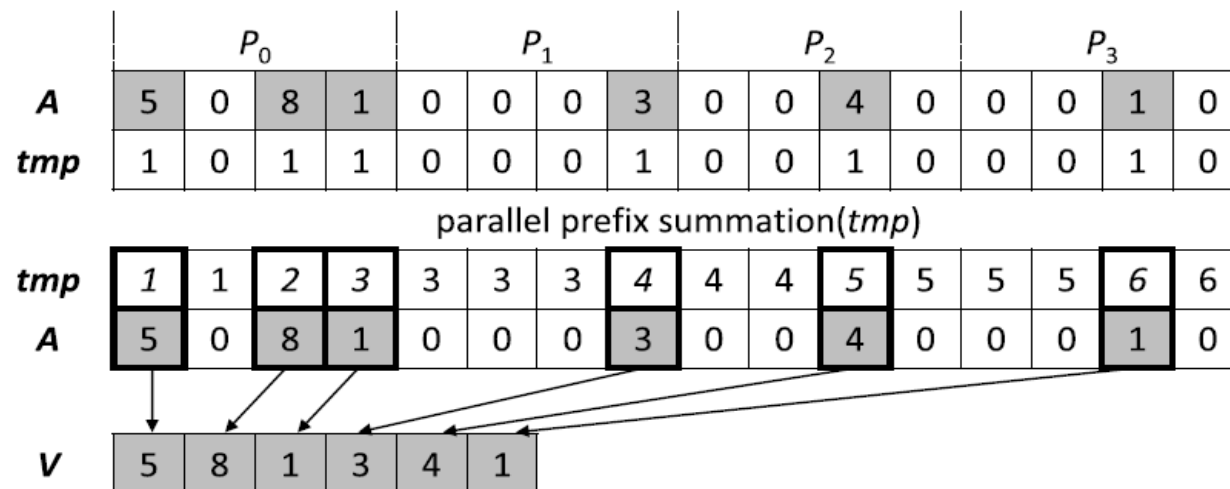
Sparse array compaction

- Assume you have a one-dimensional array A where most entries are zero
- Goal: compact all non-zero elements while preserving their order
- We can represent the array in a more memory-efficient way by only storing the values of the non-zero entries (in V) and their corresponding coordinates (in C).



Sparse array compaction on a PRAM

- We can use a parallel prefix approach using $p = n/\log(n)$ processors
 - We generate a temporary array (tmp) with $tmp[i] = 1$ if $A[i] \neq 0$ and $tmp[i] = 0$ otherwise. We then perform a parallel prefix summation on tmp . For each non-zero element of A , the respective value stored in tmp now contains the destination address for that element in V
 - We write the non-zero elements of A to V using the addresses generated by the parallel prefix summation. The respective coordinates can be written to C in a similar way



- $$C(n) = T(n,p) \times p = O(\log n) \times O(n/\log(n)) = O(n)$$

Limits of the PRAM model

- Unrealistic memory access
 - It assumes that all processors have uniform, instantaneous access to a single shared memory.
- Perfect lockstep synchronization
 - In practice achieving and maintaining such global synchronization introduce significant overhead
- No communication and memory hierarchy costs
- Cost-optimal PRAM algorithms may still scale poorly on real machines processing resource

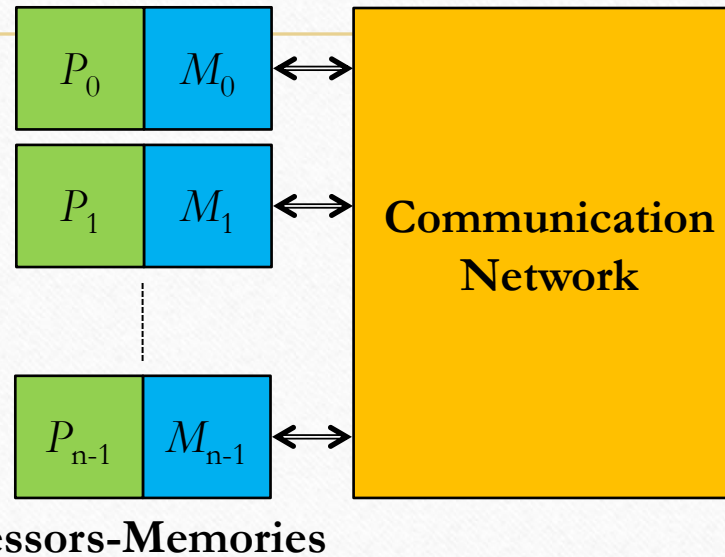
While the PRAM model provides a clean framework for designing parallel algorithms, its simplifying assumptions make it less representative of real-world systems, where memory hierarchy, communication delays, synchronization overheads, and limited resources play a crucial role

Bulk Synchronous Parallel Model

- The Bulk-Synchronous Parallel (**BSP**) model of computation was proposed by Leslie G. Valiant as a unified framework for the design, analysis, and programming of general-purpose parallel systems.
- It can deliver both scalable performance and architecture independence
- It is not just a theoretical model, it can also serve as a reference paradigm for parallel programming
- It consists of three parts:
 - A collection of processor-memory components
 - A communication network that can deliver point-to-point messages among processors
 - A facility for global synchronization (**barrier**) of all processors

Main goal: separate computation, communication, and synchronization so that algorithm design becomes simpler and more predictable.

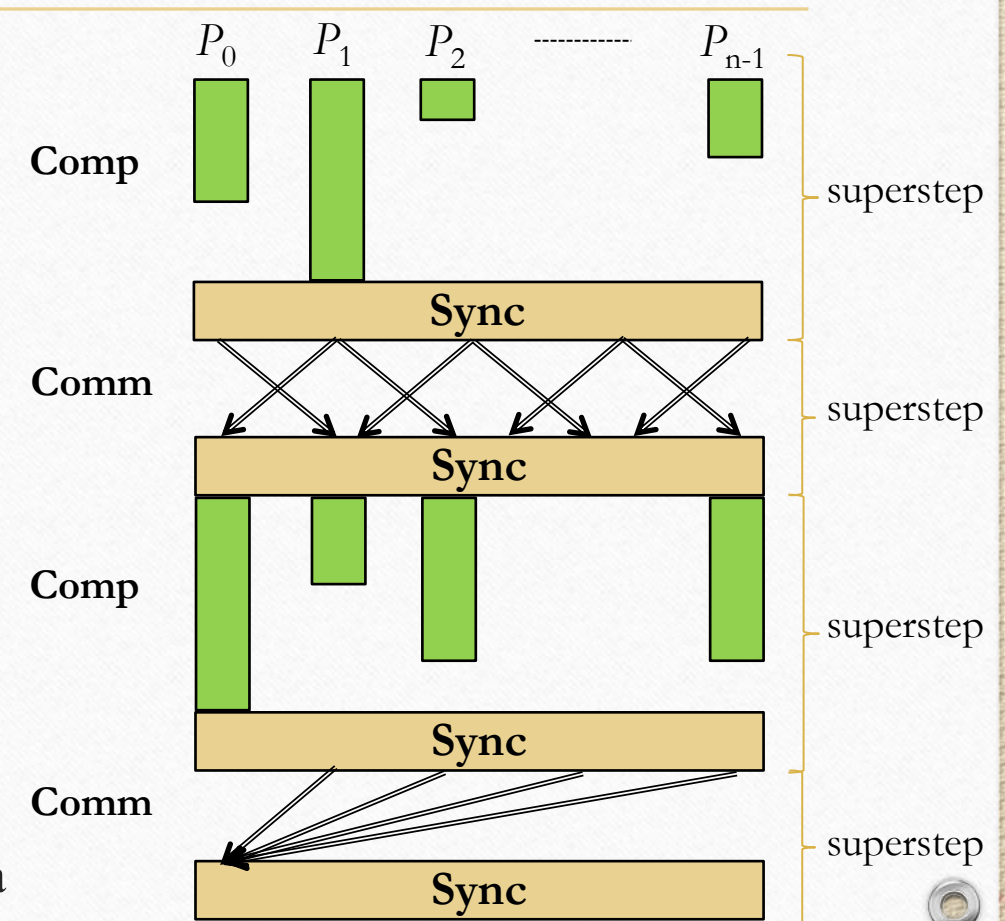
BSP Computer



- n processors $P_0, \dots, P_{n-1}$ each with its own memory (a distributed memory multiprocessor).
- A processor can access its local memory directly during computation. Data owned by another processor must be obtained through explicit communication over the network
- Communication between processors happens via explicit messaging (message-passing model)

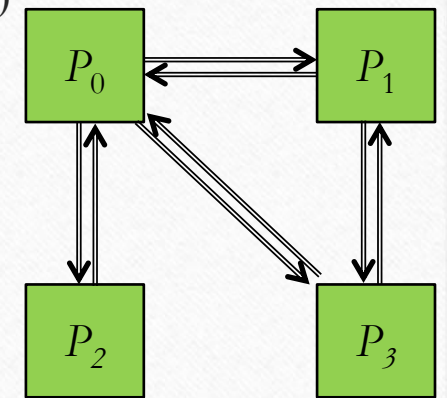
BSP Algorithm

- A BSP algorithm consists of a sequence of **supersteps**
- A superstep consists of
 - **computation, communication, or computation and communication** steps
 - so-called **mixed superstep** in case both are executed
 - **a global barrier** synchronization (*bulk synchronization*) to ensure that all computation and communication from the superstep have completed
- A **computation superstep** focuses on local processing tasks (e.g., floating point operations)
- A **communication superstep** focuses on exchanging data among processes (e.g., data words transferring)
- A **mixed superstep** may include both local processing and data exchange within the same superstep



h-relation and communication cost

- An *h*-relation is a communication superstep in which, for every processor, the number of data words sent and received is at most h
- The cost of an *h*-relation is $T(h) = h \cdot g + l$
 - g (gap) is the per-word communication cost, and l (latency) is the global synchronization time
 - Both are usually expressed in FLOPS (i.e., the time is multiplied by the processor's FLOP rate)
 - **Note that g and l depend on the number of processors (p)**
 - $l > 0$ includes the costs of the global synchronization plus all fixed costs for ensuring that all data have arrived at the destination, and the start-up of the communications
 - Approximate values for g and l on a given parallel computer can be obtained by measuring execution times for a range of full *h*-relations, varying h and p
 - A full *h*-relation is an *h*-relation where each processor sends and receives exactly h data words



(non-full)
3-relations

Computation superstep cost

- The cost of a computation superstep is $T_{comp}(w) = w + l$
 - w is the amount of work defined as the **maximum** number of operations performed in the superstep by any processor
 - Processors performing fewer than w operations remain idle until the barrier
- The cost of a mixed superstep is $T_{mix}(w) = w + h \cdot g + l$
 - The same l measure is used for all types of superstep

Therefore, BSP superstep costs are driven by three quantities: local work w , communication volume h , and synchronization latency l

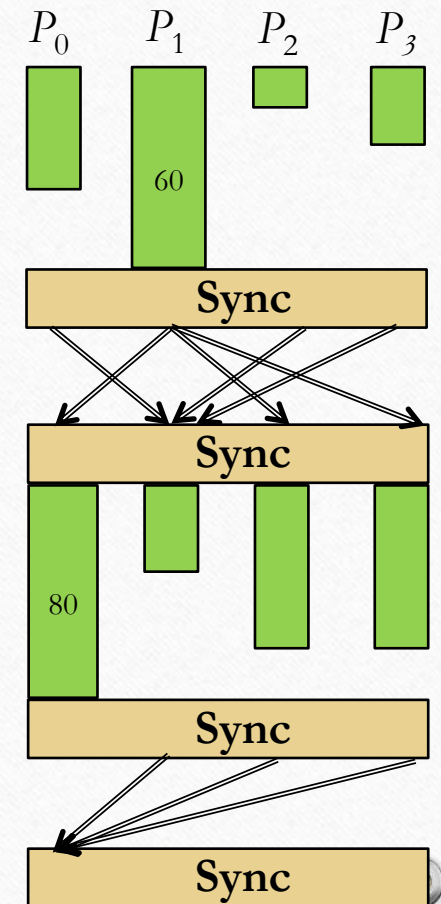
Cost of a BSP algorithm

- The BSP algorithm cost is given by the following general expression

$$C_{BSP} = a + b \cdot g + c \cdot l$$

- The expression is obtained by adding the costs of all the algorithm's supersteps, the coefficients a , b , and c collect, respectively, computation, communication, and synchronization contributions
- Example (side figure): $p=4$, $g=4$ FLOPS, and $l=20$ FLOPS
 - In the first comp. superstep P_1 takes 60 FLOPS, while in the second one P_0 takes 80 FLOPS
 - In the first comm. superstep P_1 sends/receives 10 words to/from all other processors
 - In the second comm. superstep P_0 receives 10 words from all other processors

$$\begin{aligned} C_{BSP} &= (60 + l) + ((3 \cdot 10) \cdot g + l) + (80 + l) + ((3 \cdot 10) \cdot g + l) \\ &= 140 + 60 \cdot g + 4 \cdot l \\ &= 140 + 60 \cdot 4 + 4 \cdot 20 = 460 \text{ FLOPS} \end{aligned}$$



Dot product BSP algorithm example

- $a = [a_0, \dots, a_{n-1}]^T$ and $b = [b_0, \dots, b_{n-1}]^T$ the dot product is $\alpha = a^T \cdot b = \sum_{i=0}^{n-1} a_i b_i$

- Example $n=10, p=3$

3 -1 2 5 -3 1 4 -6 8 10

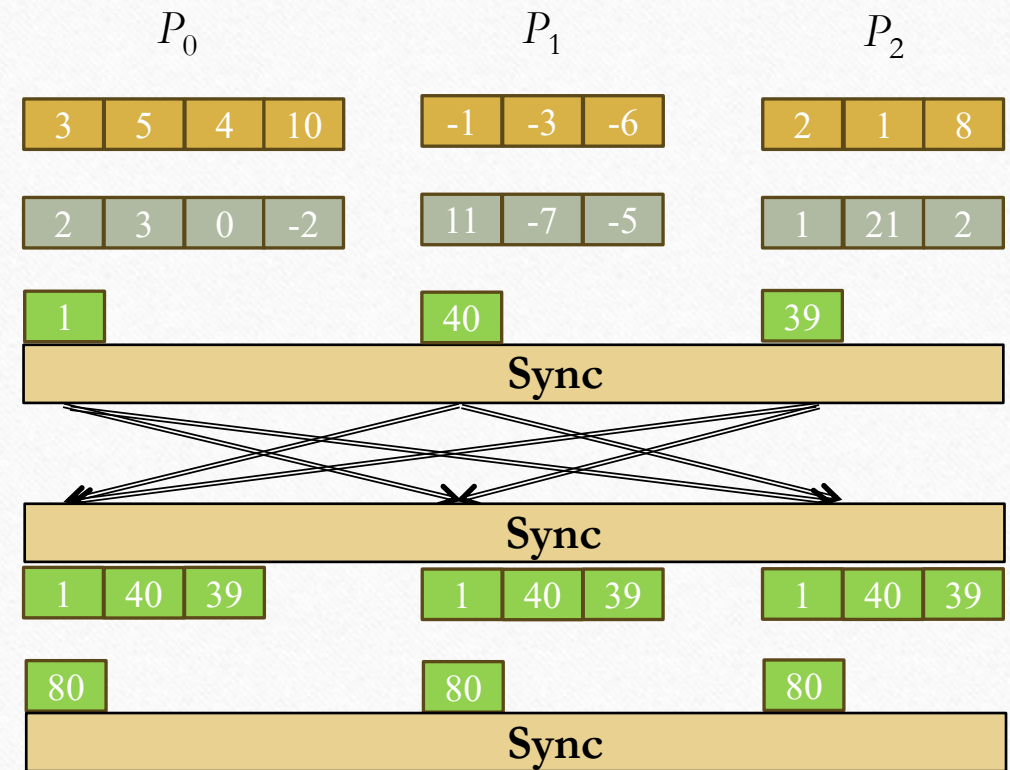
2 11 1 3 -7 21 0 -5 2 -2

$\alpha = 80$

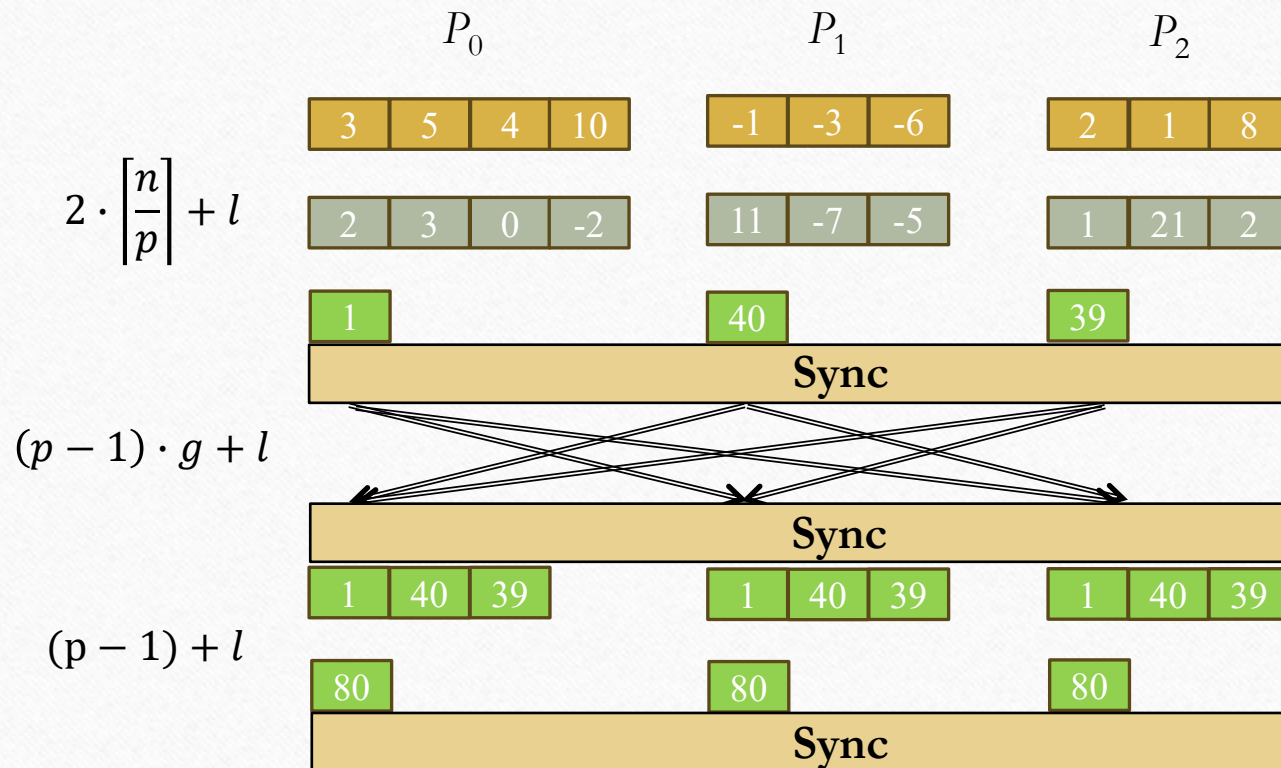
- Let's consider p processors and a **cyclic distribution** of the two arrays a, b :

$$a_i, b_i \rightarrow p_{i \bmod p} \quad i \in [0, n[$$

- Each processor computes a local partial dot product over the n/p elements
- 3 supersteps; in the end all p processor will compute α



Dot product BSP algorithm example (cont.)



$$C_{dot\ product} = \left[2 \cdot \left\lceil \frac{n}{p} \right\rceil + (p-1) \right] + (p-1) \cdot g + 3l$$

If $n \gg p$ and is large enough such that local computation dominates the BSP overhead terms $(p-1)g$ and $3l$, then the speedup is $S(p) \approx p$

BSP algorithm: GEMM

- Assumption: matrices are distributed evenly over p processors
- Processors are organized in a logical grid of size $\sqrt{p} \times \sqrt{p}$
- A, B, and C have size $N \times N$. Initially, each processor holds $\frac{N^2}{p}$ entries of A and $\frac{N^2}{p}$ entries B
 - Each sub-block of A and B has size $\frac{N}{\sqrt{p}} \times \frac{N}{\sqrt{p}}$. For simplicity, let's assume $N \bmod \sqrt{p} = 0$
- At the end of the computation, each processor will hold $\frac{N^2}{p}$ entries of C

BSP algorithm: GEMM (cont.)

- BSP algorithm: 2 supersteps
 - **Communication superstep**: processors exchange the A and B blocks needed to compute their local block of C
 - The total words communicated per processor is about $2 \cdot (\sqrt{p} - 1) \times \frac{N^2}{p} \approx 2 \frac{N^2}{\sqrt{p}}$, hence $h \approx 2 \cdot \frac{N^2}{\sqrt{p}}$
 - **Computation superstep**: each processor multiplies the received data locally and computes its $\frac{N^2}{p}$ entries of C
 - The cost is about $2 \cdot \frac{N^2}{p} \cdot N = \frac{2 \cdot N^3}{p}$ FLOPs
- BSP algorithm cost: $C_{BSP}(N, p) = \frac{2 \cdot N^3}{p} + g \cdot 2 \cdot \frac{N^2}{\sqrt{p}} + 2 \cdot l$.
- The **speedup** is $S(p) = \frac{T(1)}{T(p)} = \frac{2 \cdot N^3}{\frac{2 \cdot N^3}{p} + g \cdot 2 \cdot \frac{N^2}{\sqrt{p}} + 2 \cdot l}$
- For fixed p , the asymptotic efficiency as N increase is $\lim_{N \rightarrow \infty} E(p) = 1$

BSP model summary

- A BSP abstract machine is a distributed memory system represented as $BSP(p, r, g, l)$, where
 - p is the number of processors, r is the local computing rate expressed in FLOPS, g is the communication cost per data word (in FLOP time unit), and l is the global synchronization cost (in FLOP time unit)
- The communication network is a **black box** and its behavior is abstracted through the parameters g and l , rather than low-level network details
- Parallel programs on a BSP abstract machine are expressed as a consecutive set of supersteps where, at the end of each superstep, there is a **global synchronization barrier** among all processors
- The cost of an algorithm on the BSP machine is given by summing the cost of each superstep

BSP design implications

- A **structured approach** to thinking about parallelization. BSP organizes a parallel program into supersteps, which improves clarity and provides a simple cost model
- **SPMD-style programming** (Single Program, Multiple Data). A single program is executed by all processors, each working on different local data
- **Balanced computation**, since the cost depends on the most loaded processor
- **Balancing communication**. Good BSP algorithms avoid concentrating too much communication on a small number of processors
- **Fewer synchronization**. Since each superstep ends with a barrier, unnecessary supersteps should be avoided

Suggested Readings

- Chapter 2 Section 2.1 of the «Parallel Programming Concepts and Practice» book
- L. G. Valiant. “A bridging model for parallel computation”. Communications of the ACM, Volume 33, Issue 8, pp. 103-111, August 1990.
- «Parallel Scientific Computation: A Structured Approach Using BSP » R. H. Bisseling, Oxford University Press, 2020, (second edition)
 - More info here: <https://webspacescience.uu.nl/~bisse101/Book2/psc2.html>
- «Structured Parallel Programming – Patterns for Efficient Computation» M. McCool, A. D. Robison, J. Reinder, Morgan Kaufmann
 - Chapter 2 of this book is provided as additional material